

sat-mb-fidelity____fidelity-binomial-f-k4

An AgentMPI run analysis

Abstract

This is the binomial-tree cell of the saturation attempt, and it is the run that supplies the paper’s “at equal quality the tree wins” number: `\NFidTree` is 78, and it is this run’s 77.56s. Eight ranks contribute twelve uniquely identified items each into a 450-token merge budget — three times the items and half the budget of the `real-mb-fidelity` baseline, which is what makes this a saturation study — and it does not saturate. Retention is 1.000, all 96 identifiers reach the root, none is fabricated. The store shows why, and the answer is not that 96 items fit: the operator re-encoded, converging on a packed notation that carries an item in 14.3 bytes at the root where its own depth-1 output spent 57.7, and then it overran, emitting 698 tokens against a stated 450. The contract in this run’s fabric records `max_tokens: null` with an empty semantics string against `meta` of `{"max_tokens": 450}`, so this is the advisory regime and the budget was never checked. The measurement I find most useful here is one the aggregate hides. This run performs four structurally identical depth-1 merges: same algorithm, same depth, same 933-token prompt, 24 items in and 24 items out every time. Their outputs are 211, 269, 356 and 491 tokens — a $2.3\times$ spread in how long an operator chooses to be when nothing about its task has changed. Any arithmetic that predicts saturation from a per-item token cost has to survive that variance, and in the compressible regime it does not.

1 What this run is

property	value
run	sat-mb-fidelity__fidelity-binomial-f-k4
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	broker $\times$ 8, worker $\times$ 17
events	86
wall time	77.57 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	1.86×	total busy time over wall time
parallel efficiency	23.2%	achieved parallelism per rank
idle fraction	76.8%	rank-seconds paid for and unused
peak ranks busy	4	of 8 in the world
serial fraction of active time	49.2%	time with exactly one rank busy
load imbalance	3.54×	slowest rank’s busy time over the mean
coordination share	26.9%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	17	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	24.01 / 27.46 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	3,004	0 deferred under rendezvous
tokens in / out	6,318 / 2,903	prompt and completion
cost	\$0.0625	0.0215 \$/kTok out

3 What the analysis notices

- **[critical]** At least one rank waited 5s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- **[note]** Parallel efficiency is 23.2%: 76.8% of the rank-seconds paid for went unused.
- **[note]** Load imbalance is 3.54×: the slowest rank was busy far longer than the mean.
- **[ok]** 17 rank reattachment(s) occurred, up to incarnation 6: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	63.83	59.4	3	3	0	3	0	0.0	finalized
1	0.00	0.0	0	0	1	0	398	0.0	finalized
2	22.15	79.5	1	1	1	1	376	0.0	finalized
3	0.00	0.0	0	0	1	0	398	0.0	finalized
4	37.09	35.2	2	2	1	2	525	0.0	finalized
5	0.00	0.0	0	0	1	0	398	0.0	finalized
6	21.00	77.4	1	1	1	1	511	0.0	finalized
7	0.00	0.0	0	0	1	0	398	0.0	finalized

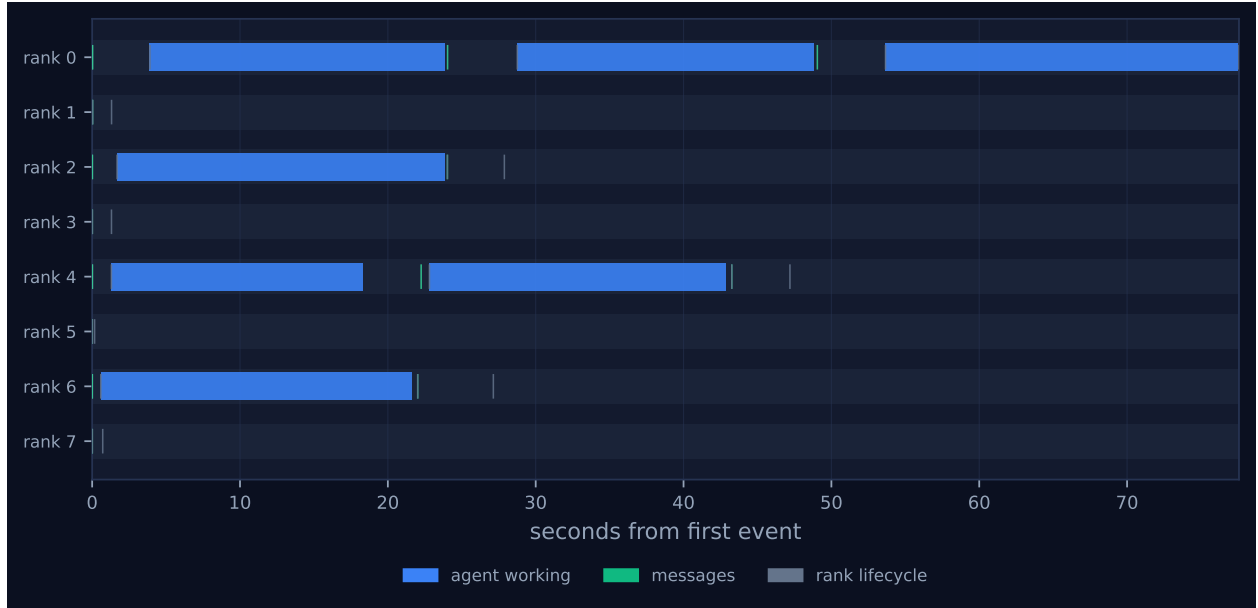


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

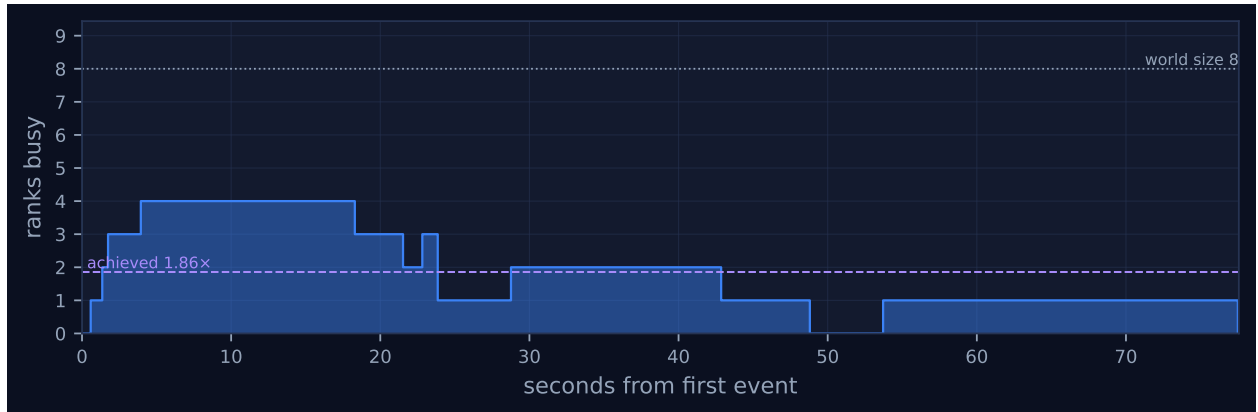


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

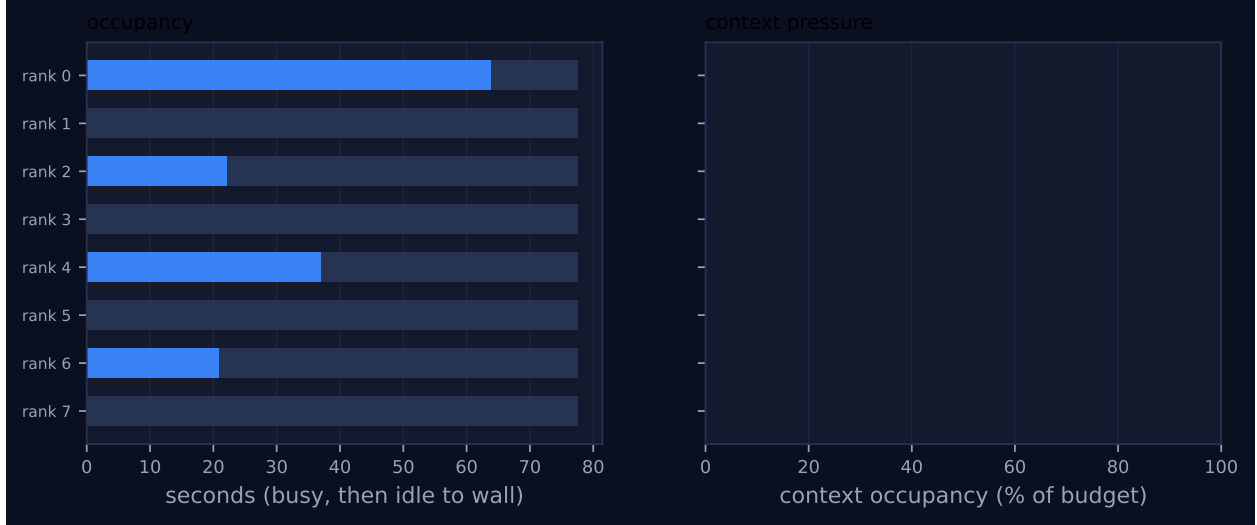


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	binomial	–	8	8	3	3	7	7	2,900	77.56	77.56	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is the textbook binomial tree in three bands, and it is the only cell in this campaign with substantial concurrency. Ranks 0, 2, 4 and 6 all hold bars together from about $t = 1$ s to $t = 22$ s; ranks 0 and 4 hold bars from about $t = 25$ s to $t = 48$ s; rank 0 holds the last bar alone to 77.56 s. Peak ranks busy is 4 of 8, achieved parallelism $1.86\times$, and the serial fraction of active time is 49.2% — half the run is the necessarily serial upper levels. Ranks 1, 3, 5 and 7 emit a tick each inside the first 40 ms and then nothing: they are the tree’s leaves, their `coll.reduce` records `fold_depth`: 0, and their idleness is why load imbalance reads $3.54\times$ and 76.8% of purchased rank-seconds went unused. Both are structural at $p = 8$ and neither is a defect.

The bar widths are strikingly uniform and the outputs behind them are not. Seven merges at 21.00, 17.03, 22.15, 19.96, 20.06, 20.02 and 23.85 s — a range of $1.4\times$ — on prompts of 933, 933, 933, 933,

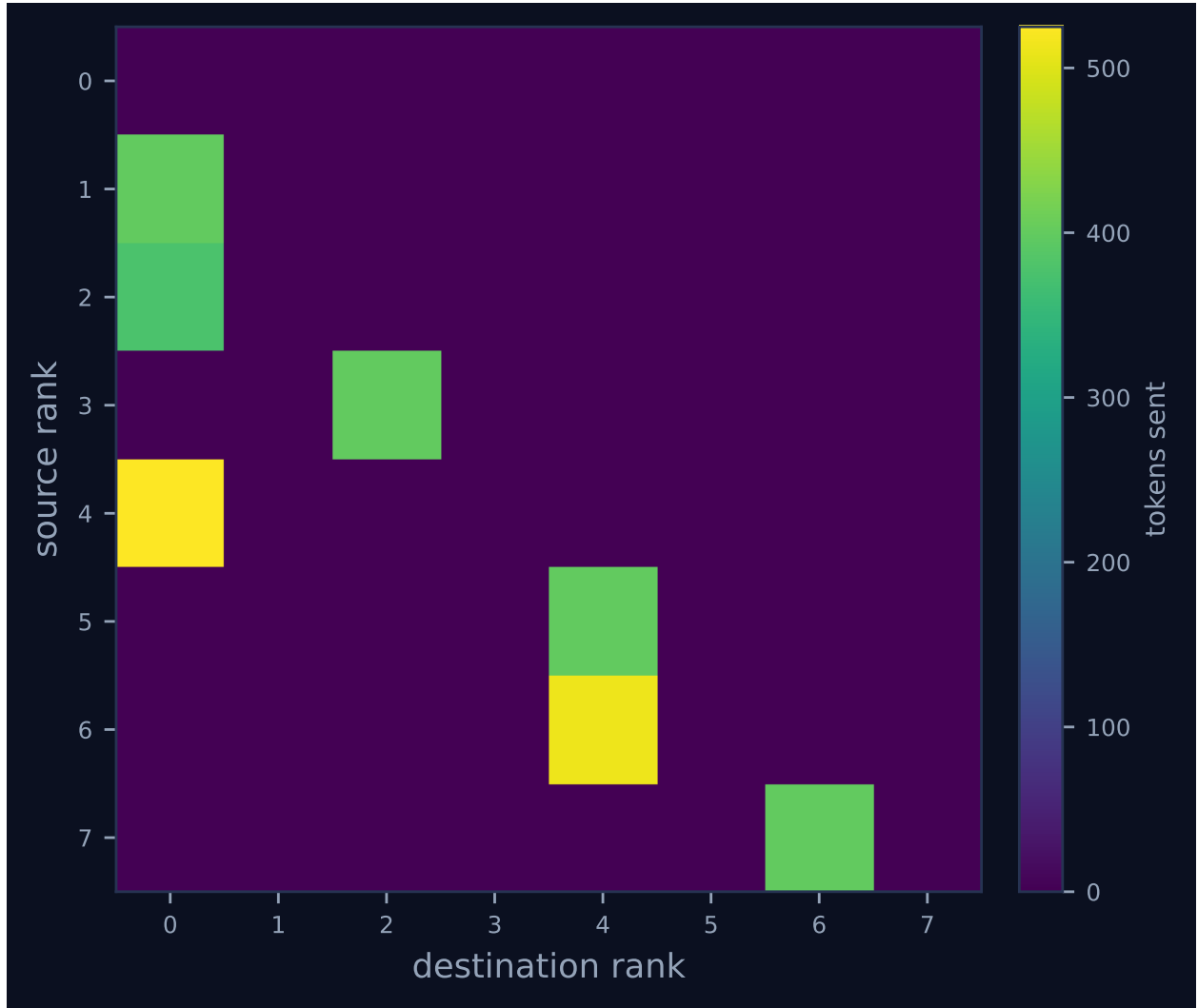


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

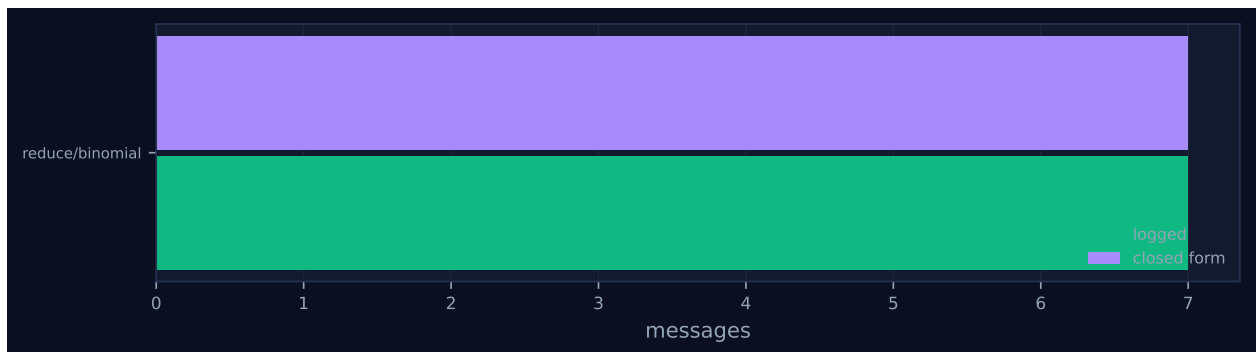


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

887, 694 and 1,005 tokens, and outputs of 491, 269, 356, 211, 505, 373 and 698. Latency is nearly constant while output length varies by $2.3\times$, which says the per-call cost here is dominated by a fixed overhead rather than by generation volume; the calibration block agrees, fitting $\alpha = 21.2$ s with a crossover at 2,788 tokens, far above anything this run produced. Of the run’s total, 144.1s of generation was spread over up to four concurrent lanes to give 77.56s of wall clock, and only 17.3s went on waiting for a worker to claim a queued call, which I obtained by differencing each `broker.enqueue` $\rightarrow$ `broker.claim` $\rightarrow$ `broker.complete` triple in the log. This cell is therefore not distorted by pool starvation, unlike its chain sibling, where the same decomposition finds 1,100s of queue wait. The log’s three events after `job.finish` and the 17 reattachments are the pool recycling after the job ended; the dashboard’s rank health table shows all eight ranks red and suspected `fail_stop` for that reason, while `metrics.json` records `ok: true, failed_ranks: []` and `state: finalized` throughout.

The communication matrix, Figure 4, is the tree’s adjacency: $1 \rightarrow 0$, $3 \rightarrow 2$, $5 \rightarrow 4$, $7 \rightarrow 6$ at 398 tokens each, then $2 \rightarrow 0$ at 376 and $6 \rightarrow 4$ at 511, then $4 \rightarrow 0$ at 525, for seven messages and 3,004 tokens. The edge weights carry the compression story in miniature: a leaf ships 12 items in 398 tokens and the $2 \rightarrow 0$ edge ships 24 items in 376, so an accumulator standing for twice the content is smaller than a single leaf report. That is re-encoding visible in the fabric, not in the outputs.

7 Interpretation

Everything structural is by construction and both checks pass: seven messages against a closed form of $p - 1 = 7$, three rounds against $\lceil \log_2 8 \rceil = 3$, so `model_checks_agree` is 1 of 1 and Figure 5 shows no red diamond. What the `sat-` prefix promises has to be read out of the config, and the comparison is with `real-mb-fidelity`: same $p = 8$, same executor, same three algorithms, but `facts` raised from 4 to 12 and `merge_budget` lowered from 900 to 450, plus `kary` added at `fanin`: 4. The root must therefore render 96 items into half the space the baseline had for 32 — a sixfold increase in pressure — and the baseline’s own accumulators cost roughly 19.4 tokens per item, which would put the 450-token capacity at about 23 items. On that arithmetic 96 items should overflow it four times over.

It did not saturate, and establishing why is this campaign’s actual result. The first half of the answer is re-encoding. Counting bytes per identifier in every accumulator this run produced: 57.7, 27.4, 35.2 and 20.4 at depth 1, then 25.3 and 16.4 at depth 2, then 14.3 at the root. The final result is not prose but a packed table, `[F-0-0]100n [F-0-1]101d [F-0-2]102s ...`, with the workload letters legended in the title — exactly the behaviour `make_fact_report`’s docstring describes, and the reason the `incompressible` variant exists. The second half is that the budget was advisory. Every one of the seven `agent_calls` rows records `contract = {"name": "MergedReport", ..., "max_tokens": null, ..., "semantics": ""}` while `meta` records `{"max_tokens": 450}`. The number reached the executor as a generation hint a worker may ignore, and never reached the contract checker that would have rejected an overrun and re-issued the call. The root’s 698 tokens are 55% over the stated limit; 96 items at 14.3 bytes is about 7.3 tokens each, or 700 tokens, so fitting 96 items inside 450 would have required 4.7 tokens each and the operator was never asked to try. All seven calls show `attempt: 1` and the run reports zero contract violations, which under a null bound is uninformative rather than reassuring. The corrected experiment is `inc-mb-fidelity-inc: max_tokens: 450` on the contract, a non-empty semantics clause, high-entropy payloads that cannot be paraphrased shorter, and retention falls to 0.385 with four of eight ranks erased entirely.

The measurement that generalises beyond this campaign is the variance across the four depth-1

merges. They are as close to replicates as this experiment contains — identical algorithm, identical fold depth, identical 933-token prompt structure, 24 items in and 24 out — and they returned 211, 269, 356 and 491 tokens, 20.4 to 57.7 bytes per item. So “how many tokens an item costs” is not a property of the item in the compressible regime; it is a choice the operator makes afresh each call, varying by a factor of 2.8. That matters for two claims. It means a-priori saturation estimates cannot work here, which is why `make_tables.py` classifies capacity-boundedness empirically — “did some algorithm actually lose an item” — rather than from a token budget. And it means the clean 11.81-tokens-per-item arithmetic that predicts the `inc-mb` campaign’s 37 survivors to within one item is a property of *incompressible* content specifically: when there is no wording freedom left, the cost per item becomes a constant and the budget becomes a divisor.

Relative to its siblings this cell is the cost baseline, and with quality pinned at 1.000 the comparison is clean. All four algorithms do the same collective over the same 96 items and send the same seven messages. This one uses seven operator applications over three levels and finishes in 77.56 s at \$0.0625; the k -ary tree uses three applications over two levels and finishes in 53.07 s at \$0.0369; the flat fold puts all seven applications on the root and takes 197.62 s; the chain takes 1,376.98 s, of which 1,100 s is a worker failing to appear. That last figure is the one to be careful with, because `make_tables.py` fills `\NFidTree`, `\NFidChain`, `\NFidFlat` and `\NFidSpeedup` from whichever broker-executed block scored 1.000 last, which is this campaign; the published “17.8× latency reduction for free” is this run’s 78 s against a chain figure that is four-fifths queueing. The structural speedup this cell supports is 7/3 applications on the critical path, and the honest tree-versus-chain number from the `real-mb` baseline is $130.57 / 58.25 = 2.2\times$.

Of the flagged findings, the 23.2% parallel efficiency and the $3.54\times$ imbalance are notes and are structural for a binomial tree over eight ranks. The 17 reattachments are the broker’s normal mode. The model check is the check passing. Nothing in the list is a defect; the defect in this campaign is the unenforced budget, and no finding can see it because a null bound generates no violations.

8 Threats to this reading

The executor is real — `executors` records `broker` × 8 and `worker` × 17 — so these are agent behaviours, unlike the `fid-synth` and `fid-inc-smoke` cells whose `function` executor implements a drop model by construction. The most serious threat is that retention 1.000 here may not measure information transport at all. `make_fact_report` builds item i of rank r with throughput $100 + 7r + i$ under workload `['nominal', 'degraded', 'saturated']` `[i % 3]`, so the payload is a deterministic function of the identifier, and the packed encoding the operator converged on reproduces that rule literally: rank 0’s items are `100n 101d 102s ...` and rank 1’s are `107n 108d ....` A root that had lost a whole message could continue the arithmetic and emit the missing identifiers with correct values, and `fact_retention` — which scores only whether the identifier appears — would call it perfect. Every retention claim in this campaign is a claim about identifier survival on a guessable payload. The `-inc` campaign scores the six-digit serial and four-letter checksum as well, and reports `n_label_only: 0`; this run reports no such quantity because the compressible bench does not compute it.

Second, three of this run’s seven results are byte-identical to results in sibling cells: `56406e77f286` (505 tokens, ranks 4–7) is also the chain’s `merge:d3` and the k -ary cell’s `merge:k4:d1`; `725368554341` (373 tokens, ranks 0–3) is also the flat cell’s `merge:d3` and the k -ary cell’s other `merge:k4:d1`; and `e411144037d6` (698 tokens, all 96 items) is also the flat cell’s `merge:d7` and the k -ary cell’s `merge:k2:d2`. The four cells ran sequentially against one persistent pool of Cursor subagents, so

reuse is mechanically possible. I ran the determinism test the contract requires, and it comes out against determinism: prompt digest `20cdeba13af0` is shared between this run's `merge:d1` and the chain's and returns 491 against 490 tokens with different digests, and `edf28c1217be` is shared between this run's `merge:d1` and the flat cell's and returns 211 against 354. Identical input does not give identical output here, so determinism cannot explain the identities. What can is convergence on the canonical packed encoding: once the notation and the arithmetic rule are fixed, one string renders a given rank set, and consistently with that the identities occur only among the dense late accumulators and never among the verbose early ones. Cross-run reuse cannot be excluded from the trace.

Third, one trial per algorithm. The comparison against the k -ary cell is safe — 7 applications against 3, 77.56s against 53.07s, ratio 1.46 where the critical-path count predicts 1.5, per-call latencies within 8% of each other. The comparisons against the flat cell and the chain are less safe: flat is credible because its mechanism is measured (seven applications on one rank, 161.6s of busy time on rank 0 alone), the chain is not, for the queueing reason above.

Finally, several accounting figures mean less than they appear to. Every `msg.recv` records `admitted: false`, because the collective's internal receives pass `admit=False`, so context occupancy is 0% and high water 0 even on a rank that read 6,318 prompt tokens; this run exercises no admission pressure and says nothing about it. The \$0.0625 is modelled from \$3/Mtok in and \$15/Mtok out with `tokenizer_exact: false`, not billed. And `rank_position_correlation: 0.000` is not a measurement of positional fairness but an undefined quantity: at retention 1.000 the per-rank variance is zero, the correlation's denominator vanishes, and the code returns 0.0 by a guard.